

APUNTES DE RUST

ESTELA M.^a RODRÍGUEZ CÉSARO

INGENIERÍA TELEMÁTICA

URJC FUENLABRADA

(24/25)

ÍNDICE

TIPOS DE DATOS EN RUST.	4
• Tipos escalares.....	4
• Tipos compuestos	4
• Variables.....	4
CONTROL DEL FLUJO.....	5
• Bloques.....	5
• Funciones	5
• Condicionales: if.....	5
• Expresiones match	6
• Bucles while	6
• Bucles loop	6
• Bucles for.....	7
• Break y continue	7
• Macros (!)	7
TIPOS COMPUESTOS	8
• Arrays.....	8
• Structs	9
• Tuplas	10
• Struct-tupla	10
• Enums.....	11
• Guardas	12
GESTIÓN DE MEMORIA.....	13
• Referencias.....	13
• Transferencia de propiedad	13
○ <i>Recuperar un valor tras transferirlo</i>	14
○ <i>Clone</i>	14
• ATAJS DE MATCH	15
○ <i>If let</i>	15
○ <i>Let else</i>	15
○ <i>While let</i>	16
• Tipos Alias	16
• Constantes	17

MÉTODOS Y RASGOS.....	17
• Métodos.....	17
○ IMPL.....	17
○ Simplificación de métodos.....	18
○ Funciones asociadas: (constructor new).....	18
○ Constantes asociadas.....	19
• Rasgos.....	19
○ Implementación de rasgos.....	19
○ Uso de rasgos.....	20
○ Tipos asociados.....	21
GESTIÓN DE ERRORES.	22
• Tipos de errores.....	22
○ Panicking.....	22
• Enum Option<T>.....	22
○ Unwrap() y expect() con Option.....	22
• Enum Result<T, E>.....	23
○ Unwrap() y expect() con Result.....	23
GENÉRICOS.....	23
• Stricts genéricos.....	23
• Enums genéricos.....	23
• Funciones genéricas.....	24
• IMPL genéricos.....	24
• Constantes genéricas.....	24
• PILA genérica.....	25
• COLA genérica.....	26

TIPOS DE DATOS EN RUST.

• Tipos escalares

- **Enteros**
 - **Isize:** i8, i16, i32, i64, i128, para enteros **con** signo.
 - **Usize:** u8, u16, u32, u64, u128, para enteros **sin** signo.
- **Reales/float:** f32 y f64.
- **Booleanos (bool):** true o false.
- **Caracteres (char):** Representados en Unicode, 'r', 'a', 'α', '∞'...
- **Strings:**
 - **String:** modificables -> para crear y manipular texto dinámico.
 - **&str:** de sólo lectura. (&: PUNTERO.) Solo leer, sin cambiarlo.

Se pueden convertir de un tipo a otro, por ejemplo: x: u32 **as isize**;

• Tipos compuestos

- **Tuplas** ((i32, f64, bool)) que permiten agrupar valores de diferentes tipos.
- **Arrays** ([i32; 3]) para colecciones de valores del mismo tipo y tamaño fijo.

• Variables

palabra reservada **let:** ASIGNAR

```
let x = 5;            // Variable immutable
let mut y = 10; // Variable mutable (se puede modificar)

let z: i32 = 42; // Asignación con tipo explícito
let w = 3.14;    // Asignación con inferencia de tipo (el compilador deduce el tipo).
```

mut	se usa para hacer que una variable sea modificable.
&	para referencias inmutables.
&mut	para referencias mutables.

Ejemplo:

```
let mut x = 5;            //permite cambiar el valor de x
let r = &mut x;            // r es una referencia mutable a x
*r += 1;                // Modificamos el valor de x a través de la referencia
println!("El valor de x es: {}", x);
```

INTERPOLACIÓN:

```
println!("x: {}, y: {}", x, y);    que es lo mismo que      println!("x: {x}, y: {y}");
```

CONTROL DEL FLUJO.

• Bloques

Los bloques se delimitan por llaves {...}

Cada bloque tiene un valor y un tipo: los de la última de sus expresiones

Si acaba con una sentencia (terminada con ;) entonces devuelve el **unit value ()**, de tipo ()

- `println!("x: {x}");`
- `println!("El valor de x es: {}", x);`

• Funciones

```
fn nombre_funcion(parametro1: Tipo1, parametro2: Tipo2) -> TipoRetorno {
```

```
    // Cuerpo de la función
```

```
    // Última expresión sin punto y coma es el valor de retorno
```

```
}
```

- **Return:** (devolver un valor)

Usar return puede ser útil en situaciones donde quieras salir de la función antes de llegar al final. Esto es común en estructuras de control como if o match

Esto:

```
fn suma(a: i32, b: i32) -> i32 {  
    return a + b;  
}
```

Es lo mismo que esto:

```
fn suma(a: i32, b: i32) -> i32 {  
    a + b // Retorno implícito  
}
```

• Condicionales: if

```
if condicion {
```

```
    // Bloque de código si la condición es verdadera
```

```
} else {
```

```
    // Bloque de código si la condición es falsa
```

```
}
```

OPERADORES LÓGICOS	
<code>!=</code>	Distinto que
<code>==</code>	Igual que
<code>&&</code>	AND una Y la otra
<code> </code>	OR una O la otra
<code>></code>	Mayor que
<code><</code>	Menor que
<code>>=</code>	Mayor o igual que
<code><=</code>	Menor o igual que

• Expresiones match

Es la sustitución del else if, else if, Es como un case

Estructura:

```
match valor {
    patrón1    => expresión1,
    patrón2    => expresión2,
    _ (otherwise) => expresión_predeterminada,
}
```

Ejemplo de clase:

```
fn main() {
    let input = 'x';
    match input {
        'q'                                => println!("Quitting"),
        'a' | 's' | 'w' | 'd'              => println!("Moving around"),
        '0'..'9'                            => println!("Number input"),
        key if key.is_lowercase()           => println!("Lowercase: {key}"),
        _ //sumidero                       => println!("Something else"),
    }
}
```

- El **or** dentro de los if, match, es con una sola barra |
- `key.is_lowercase()`: es un patrón de **GUARDA**.
.is_lowercase() de Rust, que devuelve true si key es un carácter en minúscula y false en caso contrario.

• Bucles while

Bucles con terminación controlada: mientras que se cumpla una condición.

Ejemplo:

```
fn main() {
    let mut i = 0;
    while i < 10 {
        i += 1;
        println!("i = {}", i);
    }
}
```

• Bucles loop

Da vueltas hasta que se ejecuta break. (Para bucles que puedan ser infinitos).

<pre>fn main() { let mut i = 0; loop { i += 1; if i > 100 { break; } } }</pre>	<pre>fn main() { let mut i = 0; let loop_result = loop { i += 1; if i > 10 { break i*2; } println!("i = {}", i); }; println!("loop_result = {}", loop_result); }</pre>
---	---

- `break i*2` hace que la expresión del bucle loop se sustituya por el valor `i*2` cuando `i > 10`

• Bucles for

Para iterar (**recorrer**) elementos.

```
fn main() {  
    for num in 0..5 { //desde el cero hasta el 5 SIN INCLUIR  
        println!("{}", num);  
    }  
}
```

- Para que el rango incluya el extremo derecho: **0..=5**
- Para recorrer de final a principio (inversa): *for num in (0..5).rev()* {

```
fn main() {  
    for ch in "Hello".chars() { // escribe por línea cada carácter del &str  
        println!("{}", ch);  
    }  
}
```

• Break y continue

Cuando hay bucles anidados (uno dentro de otro), se pueden usar etiquetas para indicar de cuál se quiere salir (break out).

Continue sirve para inmediatamente pasar a la siguiente vuelta de un bucle

```
1 fn main() {  
2     'rows: for x in 0..5 {  
3         'cols: for y in 0..5 {  
4             println!("x = {}, y = {}", x, y);  
5             if x + y >= 4 {  
6                 continue 'rows;  
7             }  
8         }  
9     }  
10 }
```

• Macros (!)

Son instrucciones que generan o modifican código en tiempo de compilación.
Se escriben con ! al final.

Ejemplos importantes:

- **println!** se usa para imprimir texto en la consola.
- **vec!** se usa para crear un vector (una lista de valores) de manera rápida.

```
fn main() {  
    let numeros = vec![1, 2, 3, 4, 5];  
    println!("{}", numeros); // Imprime "[1, 2, 3, 4, 5]"  
}
```

- **todo!()** para marcar que esa parte de código no está implementada.
- **format!** funciona como println! pero no muestra la salida por pantalla, te devuelve un String

```
let cadena = format!("Hola, {}!", "Alejandro");  
println!("{}", cadena); // Imprime "Hola, Alejandro!"
```

TIPOS COMPUESTOS

- **Arrays**

Colección de elementos de tipo y tamaño fijos. Tamaño inmutable.

Sintaxis: [tipo de datos, tamaño]

```
let numeros: [i32; 3] = [1, 2, 3]; //Array de enteros de tamaño 3
```

El índice, (**la posición**), es de tipo **usize**.

RECORRER UN ARRAY:

```
fn main() {  
    let primes = [2, 3, 5, 7, 11, 13, 17, 19];  
    for prime in primes {  
        for i in 2..prime {  
            assert_ne!(prime % i, 0); //not equal  
        } //lo contrario que assert_eq!  
    }  
}
```

ORDENAR UN ARRAY:

.sort() ordena de menor a mayor:

```
fn main() {  
    let mut array = [3,5,2,1];  
    println!("{array:?}"); // imprime [3, 5, 2, 1]  
    array.sort();  
    println!("{array:?}"); // imprime [1, 2, 3, 5]  
}
```

RODAJAS:

Contienen un puntero a otro array, para tener solo un trozo.

Ejemplo:

```
fn main() {  
    let array = [1, 2, 3, 4];  
    let slice: &[i32] = &array;  
    // rodaja de todos los elementos del array literal  
    let subslice: &[i32] = &slice[0..2];  
    // rodaja de sólo los dos primeros elementos  
    println!("{array:?}, {slice:?}, {subslice:?}");  
}
```


- **Structs**

Cada struct es un nuevo tipo.

SINTAXIS: { x: tipo, y: tipo }

```
struct Point {  
    x: i32,  
    y: i32,  
}
```

```
fn main() {  
    let p = Point { x: 1, y: 2 };  
    let q = Point { x: 4, ..p };    //..p significa "lo que falta lo coges de p"  
}  
  
fn main() {  
    let p = Point { x: 1, y: 2 };  
    println!("{}", p.x);          //el operador . significa coger ESA PARTE.  
    println!("{}", p.y);  
}
```

EJEMPLO MÁS REAL:

```
struct Person {  
    name: String,    //no es &str, que no se puede modificar. String SI  
    age: u8,  
}  
  
//esta función es como el escribir barco, pasando la variable por REFERENCIA.  
fn describe(person: &Person) { //puntero a un valor de tipo person  
    println!("{}", person.name, person.age);  
}  
  
fn main() {  
    //String::from me permite construir un tipo string a partir de un literal  
    //es como si pasáramos Peter de tipo &str a String que es modificable.  
    let mut peter = Person { name: String::from("Peter"), age: 27 };  
    describe(&peter); //& porque en vd es un puntero a la variable peter  
    //saco la dirección de memoria (sacar el puntero)  
  
    peter.age = 28; //porque peter es mutable  
    describe(&peter);  
  
    //SINTAXIS  
    let name = String::from("Avery");  
    let age :u8 = 39;  
    let avery = Person { name, age }; // fíjate: las variables se llaman como los campos  
    describe(&avery);  
  
    let jackie = Person { name: String::from("Jackie"), ..avery };  
    describe(&jackie);  
}
```

- **Tuplas**

A las tuplas no se les puede poner un nombre como tal.
Almacenan valores de diferentes tipos.

```
fn main() {  
    let p = (1, "hola");           //(tipo entero, tipo &str) es decir inmutable  
    println!("p.0: {}", p.0);    //se accede a cada parte con la variable .0 .1 (.parte)  
    println!("p.1: {}", p.1);  
}
```

TUPLA VACÍA => () (es el unit value de tipo unit type).
esto es como un procedure.

```
fn prints_but_returns_nothing(data: &str) -> () { //en rust () no es necesario.  
    println!("passed string: {}", data);  
}
```

DESESTRUCTURAR TUPLAS.

```
fn print_tuple(tuple: (i32, i32)) {  
    let left = tuple.0; //me declaro una var inmutable y me guardo una parte  
    let right = tuple.1; // aquí lo mismo con la otra parte  
    println!("left: {left}, right: {right}");  
}  
// realmente esto es una mierda  
// esto mejor  
fn print_tuple(tuple: (i32, i32)) {  
    let (left, right) = tuple;  
    println!("left: {left}, right: {right}");  
}
```

LO MISMO CON LOS STRUCTS.

- **Struct-tupla**

```
struct Point(i32,i32);
```

```
fn main(){  
    let p = Point(1, 2);  
    println!("{}", p.0);  
    println!("{}", p.1);  
}
```

• Enums

Enumera los posibles valores que puede tener un tipo.

Puedo tener un struct con una parte de tipo enumerado.

```
enum Shape {           //El tipo del enumerado es figura, el propio shape
    Square,
    Circle,
    Rectangle,
    Triangle,
}
fn main() { // PARA ACCEDER ::
    let shape = Shape::Rectangle;
}
```

ENUMS CON VALORES

```
enum Movement { //son struct-tupla
    Right(i32),           //DARLE UN VALOR ENTERO con signo AL ENUMERADO
    Left(i32),
    Up(i32),
    Down { speed: i32, excitement: u8 }, // es un struct como siempre.
}
fn main() { //:: para detallar qué valor concreto quieres.
    let movement = Movement::Left(12);
    //en la const movement almacena un valor del enumerado(el valor entero)
    let movement = Movement::Down { speed: 12, excitement: 5 };
    //no puedo quitar el let porque no tiene mut
} //creo un valor del enumerado igual pero con el struct, esto machaca el valor anterior.
```

USO DE MATCH CON UN ENUM

```
enum Movement {
    Right(i32),
    Left(i32),
    Up(i32),
    Down { speed: i32, excitement: u8 },
}
// let m = Movement::Up(7);           | llega: Movement::Up(7)
//check_movement(m);                  v
fn check_movement(movement: Movement) { //COMO CREAR EL MOVIMIENTO.
    match movement { // => es como el : de pascal
        Movement::Up(desplazamiento) => { // desplazamiento = 7.
            println!("Se ha movido hacia arriba con desplazamiento {}", desplazamiento);
        } //EN CASO DE UP: al entero le llamo desplazamiento
        Movement::Down { speed, excitement } => {
            println!("Se ha movido hacia abajo con velocidad {}", speed);
        } //EN CASO DE DOWN
        _ => { // el _ es un sumidero
            println!("Se ha movido hacia la derecha o hacia la izquierda");
        } //EN OTRO CASO (otherwise)
    }
}
```

- Guardas

Una guarda es un condicional sobre un caso concreto. Una condición añadida.

```
enum Shape {           //figura (struct-tupla)
  Circle(i32),
  Rectangle(i32, i32),
}
fn check_shape(shape: Shape) {
  match shape {
    // radio < 10 NO ENTRA POR ESTA RAMA: (LA CONDICIÓN AÑADIDA)
    Shape::Circle(radius) if radius > 10 => { // se ha creado la variable radius
      println!("It's a BIG circle, with radius {}", radius);
    }
    _ => { //EN OTRO CASO:
      println!("Try a big circle instead");
    }
  }
}
```

COMBINACIÓN DE PATRONES

```
enum Shape {
  Circle(i32),
  Rectangle(i32, i32),
  Square(i32),
}
fn test_shape(shape: Shape) {
  match shape { // OR en un match solo una |
    Shape::Circle(size) | Shape::Square(size) => {
      //la variable tiene que ser la misma (size)
      println!("Shape has single size field {}", size);
    }
    _ => { // otherwise.
      println!("Not a circle, or a square");
    }
  }
}
```

GESTIÓN DE MEMORIA.

• Referencias

Las referencias permiten al propietario de un dato prestárselo a otras partes del código, conservando la propiedad (como el propietario que alquila un piso)

```
fn main() {
    let mut a = 'A';    // a es el dueño del valor 'A'
    let mut b = 'B';    // b es el dueño del valor 'B'
    let mut r: &char = &a; // r es una referencia que apunta al valor que posee a
    let r2: &char = &a; // r2 también, son referencias compartidas
    println!("{}", r); // el dueño puede seguir usando su valor, lo comparte con r y r2
    // a = 'a';        // pero no puede modificarlo porque lo tiene prestado
    println!("{}", *r); // A través de r se accede al contenido apuntado con *r
    r = &b;            // r ahora apunta al valor que posee b
    // a = 'a';        // ¿Por qué esta línea no compila?
    // b = 'b';        // ¿Por qué esta otra línea tampoco no compila?
    println!("{}", *r); // *r es el valor referenciado por r: 'B'
    println!("{}", *r2);
    a = 'a';           // Ahora sí compila porque ya no hay referencias activas
    b = 'b';           // Ahora sí compila porque ya no hay referencias activas
    // *r = 'X';       // ¿Por qué no compila?
}
```

• Transferencia de propiedad

1. **Propiedad única:** cada valor tiene un propietario único. Si ese propietario se transfiere, el valor deja de estar accesible desde el dueño anterior.
2. **Movimiento de propiedad en funciones:**
 - Al pasar un valor a una función sin &, la propiedad se transfiere a la función. Esto significa que el valor original ya no puede usarse después de la transferencia.
3. **Movimiento entre variables:**
 - Al asignar un valor de una variable a otra sin &, la propiedad se mueve

```
fn print_string(t: String) {    //NO LLAMA AL STRING CON &
    println!("The string is {t}");
}    // Cuando termina la función su valor es destruido

fn main() {
    let s = String::from("Hello 🤖");    //VARIABLE INMUTABLE DE TIPO STRING
    //s es la propietaria de ese valor, de momento
    print_string(s); //ahora el propietario es la función
    // print_string(s); // s ya no es propietaria
}

fn main() {
    let s = String::from("Hello 🤖");
    let s2 = s;    // aquí la propiedad de s se ha movido/transferido a s2
    // print_string(s); AQUÍ PETA PORQUE YA NO ES DE S
    print_string(s2); // ahora se mueve/transfiere la propiedad a print_string
    // print_string(s2); // y por eso aquí s2 deja de poder usarse
}
```

○ Recuperar un valor tras transferirlo

Si se quiere que una función devuelva un valor que mantenga después de la transferencia, la función debe retornar el valor.

(CON EL RETURN SE RECUPERA).

```
fn main() { // si s no fuese mutable, no furula
    let mut s = String::from("Hello 🤖");
    s = print_string(s);
    // s transfiere la propiedad a la función y la recupera al retornar
    println!("{}", s); // s sigue existiendo
}

fn print_string(mut t: String) -> String {
    t.push_str(" buddy"); // añade detrás de "Hello 🤖"
    println!("The string is {t}");
    return t; // Se transfiere la propiedad de vuelta al llamante
} // Cuando termina NO se destruye el valor de t, t ya no es el propietario
```

Recordatorio: Con referencias no se pierde la propiedad y no hace falta devolver el parámetro desde la función para recuperar la propiedad

○ Clone

```
fn main() {
    let s = String::from("Hello 🤖");
    let s2 = s.clone(); // hacemos una copia de s
    // ahora hay 2 String distintos, con sus respectivos propietarios s y s2
    print_string(s2); // aquí la propiedad de s2 se ha movido/transferido al
    // parámetro formal t de print_string
    print_string(s); // pero s sigue siendo propietario de su propio String
}

fn print_string(t: String) { // Ojo: no nos pasan &t, nos transfieren el valor
    println!("The string is {t}")
} // Cuando termina la función termina el ámbito de t y su valor es destruido
```

CLONE AUTOMÁTICO ES LO DE SIEMPRE CON VARIABLES.

```
fn main() {
    let x = 42;
    let y = x; // se copia el valor del entero x en y, x no pierde la propiedad de su valor
    println!("x: {x}"); // por lo que x sigue existiendo
    println!("y: {y}"); // e y existe, tiene una copia del valor que tiene x
}
```

Añadiendo esta línea no hay que poner .clone() todo el rato, es por defecto:

`#[derive(Copy, Clone, Debug)]`

- ATAJOS DE MATCH

Recordamos el **match con un enum** y **las guardas**.

```
enum Shape {
  Circle(i32),
  Rectangle(i32, i32),
}

fn describe_shape(shape: Shape) {
  match shape {
    Shape::Circle(radius) if radius > 10 => println!(" Circle with radius {}",
radius),
    Shape::Circle(radius) => println!("Circle with radius {}", radius),
    Shape::Rectangle(width, height) => println!("size {}", width, height),
  }
}
```

- If let**

Uso: match para cuando solo quiero analizar un caso.

Sintaxis: se lee de derecha a izquierda.

Ejemplo:

```
enum Shape {
  Circle(i32),
  Rectangle(i32, i32),
}

fn test_shape(shape: Shape) {
  //si me llega en la figura(shape) exclusivamente un círculo
  if let Shape::Circle(radius) = shape { // se ha creado la variable radius
    println!("Shape is a Circle with radius {}", radius);
  }
}
```

- Let else**

Uso: cuando solo quiero analizar un caso, pero además con un “sumidero” SIN IF

Sintaxis: `let Shape::Circle(radius) = shape else { ... };`

Ejemplo:

```
enum Shape {
  Circle(i32),
  Rectangle(i32, i32),
}

fn test_shape(shape: Shape) {
  let Shape::Circle(radius) = shape else {
    println!("I only like circles"); //esto se ejecuta en caso de que NO sea el círculo.
    return;
  }; // a partir de esta línea se ha creado la variable radius
  println!("Shape is a Circle with radius {}", radius); // esto si SÍ es un círculo.
}
```

o **While let**

Uso: como if let, pero en un bucle, repitiéndose mientras el patrón coincida.

Sintaxis: se lee de derecha a izquierda.

Ejemplo:

```
enum Shape {
    Circle(i32),
    Rectangle(i32, i32),
}
fn main() {                                // se ha creado la variable
    while let Shape::Circle(radius) = make_shape() {
        println!("Got circle, radius {}", radius);
    }    // SE VA A EJECUTAR HASTA QUE ENTRE UN RECTÁNGULO
}
fn make_shape() -> Shape {
    todo!() // hace una figura
}
```

• **Tipos Alias**

Crea un nuevo nombre de tipo para otro tipo que ya existe.

Sintaxis: type NuevoNombre = TipoExistente;

Ejemplo de clase (chustón):

```
enum CarryableConcretItem {
    Left,
    Right,
}
type Item = CarryableConcretItem;
type PlayerInventory = Mutex<Vec<Item>>;
//<> para especificar que los elementos son de tipo item. vec es un tipo de datos que
admite genericidad. Lo que significa que dentro puedo tener cualquier tipo de
elementos.
//Mutex es un mecanismo para proteger código crítico. ES UN TIPO. (me la pela)
```

Un ejemplo más sencillo de entender:

```
type Point = (i32, i32);
// Ahora, en lugar de escribir (i32, i32) cada vez, podemos usar Point
fn main() {
    let origen: Point = (0, 0);
    let destino: Point = (5, 10);
    println!("Origen: {:?}", origen);
    println!("Destino: {:?}", destino);
}
```


- **Constantes**

- **Constante:** Valor inmutable, constante en tiempo de compilación.
NO tienen una dirección de memoria

```
const BEST_NUMBER: u8 = 42;
fn main() {
    println!("The best number is: {BEST_NUMBER}");
}
```

- **Static:** Similar a const pero con vida extendida en todo el programa.
Sí tienen una dirección de memoria. (PUNTERO).

```
static BANNER: &str = "Welcome to RustOS 3.14";
fn main() {
    println!("{BANNER}");
}
```

MÉTODOS Y RASGOS.

- **Métodos**

Un método es una función asociada a un tipo de datos.

- **IMPL**

Implementar métodos a un tipo de datos.

Los métodos reciben un **parámetro especial, self**, que representa el valor de la instancia sobre la que se llama el método.

```
struct Square(f64);
impl Square {
```

Lo que sería la traducción para entender el self: Self
fn_area(square: Square) -> f64 { square.0 * square.0 }

```
    //si desde el impl quiero llamar a mi objeto (variable con vida propia SELF).
    //recibe un parámetro self de tipo SELF = SQUARE
    //aquí ( en área) recibo un objeto de mi tipo de datos (un puntero a un cuadrado)
    fn area(self: &Self) -> f64 { self.0 * self.0 } // método area
    fn double(self: &mut Self) { self.0 *= 2.0; } // método doble
    fn side (self: Self) -> f64 { self.0 } // método side
```

```
fn main() {
    let mut sq = Square(5.0); // ::método para ACCEDER
    Square::double(&mut sq); // referencia mutable que le permite modificar sq
    println!("area is {}", Square::area(&sq)); // area recibe una referencia compartida
    println!("side is {}", Square::side(sq)); // la propiedad de sq se transfiere a side, y
    // allí se destruye el valor de sq.
    // No tiene mucho sentido transferir la propiedad en el metodo side()
    // Es sólo un ejemplo para provocar que la siguiente línea no compile

    // println!("side is {}", Square::side(sq)); // por tanto sq ya no puede usarse aquí
}
```

○ Simplificación de métodos

Sintaxis simplificada para self:

&self en lugar de **self: &Self**.
&mut self en lugar de **self: &mut Self**.
self en lugar de **self: Self**.

Entonces con la simplificación:

```
struct Square(f64);
impl Square {
    fn area(&self) -> f64 { self.0 * self.0 }
    fn double(&mut self) { self.0 *= 2.0; } // Lleva ; porque no retorna nada, modifica.
    fn side (self: Self) -> f64 { self.0 }
}

fn main(){ // aquí ya utilizamos el operador . y nos olvidamos de las referencias
    let mut sq = Square(5.0); // ::método para ACCEDER
    sq.double(); // En vez de Square::double(&mut sq) escribimos sq . método()
    println!("area is {}", sq.area());
    println!("side is {}", sq.side());
}
```

○ Funciones asociadas: (constructor new)

Las funciones asociadas son funciones dentro de impl que **NO** toman self como parámetro.

La función asociada new() o constructor de tipo:

Similar a `__init__()` en Python. Es como crear un nuevo tipo.

Ejemplo:

```
pub struct Square(f64);

impl Square {
    //Función asociada new que actúa como un constructor
    pub fn new(width: f64) -> Square {
        Square(width)
    } //dentro del constructor del tipo puedo hacer lo que quiera pero dándole valor
    fn area(&self) -> f64 { //método área
        self.0 * self.0
    }
}

fn main(){
    // La llamamos como a una función normal:
    let sq = Square::new(5.0); // ME ESTÁ CREANDO UN OBJETO DE MI TIPO.
    println!("el area es {}", sq.area());
}
```

Pub -> "público"
para poder usarlo
desde otro sitio.

○ Constantes asociadas

ES PONER UNA CONSTANTE EN EL IMPL, para todo el tipo. Son útiles para propiedades constantes, como el número de lados en un cuadrado.

Ejemplo:

```
pub struct Square(f64);

impl Square {
    const NUMBER_OF_SIDES: u8 = 4;

    pub fn perimeter(&self) -> f64 {
        self.0 * (Self::NUMBER_OF_SIDES as f64) //transforma de un u8 a f64
        //sintaxis: Self en mayusc. self = Square:: constante
        // la constante le pertenece a todo el tipo (está dentro del tipo)
    }
}

fn main() {
    let sq = Square(5.0); // La llamamos como a una función normal
    // ME ESTÁ CREANDO UN OBJETO DE MI TIPO
    println!("El perímetro es {}", sq.perimeter());
}
```

• Rasgos

Un rasgo se define utilizando la palabra clave `trait`, seguida del nombre del rasgo y el bloque que contiene las firmas de los métodos que deben implementarse.

```
trait Describable {
    fn describir(&self) -> String;
}
```

Describable es un rasgo que requiere que cualquier tipo que lo implemente defina el método describir, el cual devuelve una cadena de texto.

○ Implementación de rasgos

Para que un tipo implemente un rasgo, se utiliza la palabra clave **impl** junto con el **nombre del rasgo** y **(for) tipo**.

```
struct Persona {
    nombre: String,
    edad: u8,
}
```

La estructura `Persona` implementa el rasgo `Describable` proporcionando su propia versión del método `describir`.

```
impl Describable for Persona {
    fn describir(&self) -> String {
        format!("{}", tiene {} años.", self.nombre, self.edad)
    }
}
```

○ **Uso de rasgos**

Cuando un tipo implementa un rasgo, puedo llamar a los métodos definidos en ese rasgo para las cosas de ese tipo.

```
fn main() {  
    let persona = Persona {  
        nombre: String::from("Alice"),  
        edad: 30,  
    };  
    println!("{}", persona.describir()); //llamada al método  
}
```

SALIDA: Alice tiene 30 años.

EJEMPLO DE CLASE:

<pre>trait HasArea { //Devuelve el área en m² fn area_m2(&self) -> f64; //Devuelve el área en acres. fn area_acres(&self) -> f64 { self.area_m2() / 4046.86 } }</pre>	}	DECLARACIÓN DEL RASGO
<pre>struct Square(f64);</pre>	}	CREAMOS EL TIPO
<pre>impl HasArea for Square { fn area_m2(&self) -> f64 { self.0 * self.0 } }</pre>	}	IMPLEMENTACIÓN DEL RASGO Impl rasgo for tipo
<pre>fn main() { let sq = Square(5.0); println!("{}", sq.area_m2()); }</pre>	}	LLAMADA AL MÉTODO DEL RASGO

○ Tipos asociados

Un rasgo puede tener tipos asociados, usados habitualmente para devolver valores en métodos del rasgo.

Ejemplo básico de tipo asociado en un rasgo

```
trait Identificable {  
    type ID; // Definimos un tipo asociado llamado `ID`  
    fn id(&self) -> Self::ID; // Método que devuelve el identificador del tipo `ID`  
}
```

Implementación del rasgo Identificable con tipos específicos

```
struct Persona {  
    nombre: String,  
    identificador: u32,  
}  
  
impl Identificable for Persona {  
    type ID = u32; // Especificamos el tipo `ID` como `u32`  
    fn id(&self) -> Self::ID {  
        self.identificador // LA LLAMADA  
    }  
}
```

EJEMPLO DE CLASE (DIAPOSITIVAS):

```
#[derive(Debug)]  
struct Meters(i32);  
  
#[derive(Debug)]  
struct MetersSquared(i32);
```

```
trait Multiply {  
    type Output; // tipo asociado: el que implemente el rasgo definirá un tipo  
                  concreto para Output  
    fn multiply(self: &Self, other: &Self) -> Self::Output;  
}  
  
// RASGO TIPO  
impl Multiply for Meters {  
    type Output = MetersSquared; // aquí damos valor al tipo Output  
    fn multiply(self: &Self, other: &Self) -> Self::Output {  
        MetersSquared(self.0 * other.0)  
    }  
}  
  
fn main() {  
    println!("{:?}", Meters(10).multiply(&Meters(20)));  
}
```

Sólo se puede *Implementar* un *Rasgo* para un *Tipo* si:

- El *Tipo* se declaró en este módulo o
- El *Rasgo* se declaró en este módulo

GESTIÓN DE ERRORES.

- **Tipos de errores**

Errores en tiempo de compilación: los que detecta el compilador.

Errores en tiempo de ejecución: los que ocurren una vez que el programa ha compilado y se está ejecutando.

- **Panicking**

Debido a un fallo CONTROLADO que interrumpe la ejecución del programa.

SOLO TENEMOS QUE: hacer una llamada a la macro **panic!()**

- **Enum Option<T>**

Representa una posibilidad de valor o la ausencia de él.

Para manejar de forma segura los valores que pueden estar presentes o ausentes, y ayuda a evitar errores de referencia nula en tiempo de ejecución.

```
enum Option<T> {  
    Some(T),          //valor  
    None,             //ausencia de valor  
}
```

EJEMPLO DE SU USO:

```
fn search(t: i32, a: &[i32]) -> Option<usize> {    // usize como parámetro genérico T  
    for i in 0..a.len() {                          // recorrer un array  
        if t == a[i] {  
            return Some(i);                        // si se encuentra un valor en esa posición retorna Some()  
        }  
    }  
    return None;                                    // si no lo encuentra NADA  
}  
  
fn main() {  
    match search(2, &[3,4,5]) {  
        Some(position) => println!("Encontrado en position {position}"),  
        None           => println!("Valor no encontrado")  
    }  
}
```

- **Unwrap() y expect() con Option**

Tras llamar a una función que devuelve Option:

- Si el valor devuelto es Some(T):
 .unwrap() extrae el valor de tipo T de Option y lo devuelve
- Si el valor devuelto es None:
 .expect("Oh no") hace panic y muestra el string "Oh no"

- **Enum Result<T, E>**

Usamos este tipo para devolver:

Ok(T) cuando se ha obtenido un valor de tipo T que se quiere devolver como resultado

Err(E) cuando NO se ha obtenido un valor adecuado que devolver y lo queremos comunicar.

```
enum Result<T, E> {  
    Ok(T),  
    Err(E)  
}
```

EJEMPLO EJ. 27:

```
match aplicar_operador(operator, &mut operands) {  
    Ok (()) => {}  
    Err(_) => {return Err(String::from(ERR_MSG))}
```

- **Unwrap() y expect() con Result**

Tras llamar a una función que devuelve Result:

- Si el valor devuelto es Ok(T):
 .unwrap() extrae el valor de tipo T de Result y lo devuelve
- Si el valor devuelto es None:
 .expect("Oh no") hace panic y muestra el string "Oh no"

GENÉRICOS.

Que algo sea genérico quiere decir que tiene que valer para cualquier tipo de datos.

LO ÚNICO IMPORTANTE (DISTINTO) AQUÍ ES LA **SINTAXIS**.

Los **parámetros** de los tipos genéricos van entre **< ... >**

- **Structs genéricos**

```
struct Point<T> { // <T> es un parámetro que describe un tipo concreto  
    x: T,  
    y: T,  
}  
fn main() {  
    let point: Point<i32> = Point { x: 1, y: 2 }; // Definimos T con un i32  
    let point = Point { x: 1_u32, y: 2 };         // Definimos T con un u32  
}
```

- **Enums genéricos**

```
enum Either<T, X> {  
    Left(T),  
    Right(X),  
}  
  
fn main() {  
    let alternative: Either<i32, f64> = Either::Left(123);  
}
```

• Funciones genéricas

```
/// Devuelve el valor de par o el de impar dependiendo de cómo sea n
fn pick<T>(n: i32, par: T, impar: T) -> T {
    if n % 2 == 0 { // % como el mod de pascal (EL RESTO DE LA DIVISIÓN)
        par
    } else {
        impar
    }
}

fn main() {
    println!("Devuelve un número: {:?}", pick(97, 222, 333));
    println!("Devuelve una tupla: {:?}", pick(28, ("perro", 1), ("gato", 2)));
}
```

• IMPL genéricos

```
struct Vector<T> {
    x: T,
    y: T,
}

impl<T> Vector<T> { // impl genérica
    fn new(x: T, y: T) -> Vector<T> {
        Vector { x, y }
    }
}

impl Vector<f32> { // Este bloque impl sólo proporciona magnitude() para f32
    fn magnitude(&self) -> f32 {
        ((self.x * self.x) + (self.y * self.y)).sqrt() // el modulo de un vector
    }
}

fn main() {
    let v1 = Vector::new(1.0, 1.0);
    println!("{}", v1.magnitude());
    let v2 = Vector::new(1, 1);
    // println!("{}", v2.magnitude()); // ¡ Sólo f32 tiene magnitude() !
}
```

• Constantes genéricas

```
struct Polygon<const SIDES: u8> {
    colour: u32
}

impl<const SIDES: u8> Polygon<SIDES> {
    fn new(colour: u32) -> Polygon<SIDES> { Polygon { colour } }
    fn print(&self) { println!("{}", SIDES, colour=0x{:06x}", SIDES, self.colour); }
}

fn main() {
    let triangle: Polygon<3> = Polygon::new(0x00FF00);
    triangle.print();
}
```


- PILA genérica

Pensar en hacer una pila de platos: vas añadiendo uno por encima del anterior

```
// T son los elementos de la pila y size el tamaño CONSTANTE
#[derive(Debug, PartialEq)] // quiero que sepa como comparar dos
valores de mi tipo con == o !=
struct Pila<T, const SIZE: usize> {
    a: [T; SIZE],
    cima: usize
}

impl <T: Default + Copy, const SIZE: usize> Pila<T, SIZE> {
    fn new() -> Pila<T, SIZE>{
        Pila { a: [T::default(); SIZE], cima: 0}
    }
    fn push(&mut self, value: T) -> Result <(), String> {
        if self.cima == SIZE { // esto es para ver
            return Err(String::from("Pila llena"));
        }
        //else { NO HACE FALTA POR EL RETURN
        self.a[self.cima] = value;
        self.cima += 1;
        return Ok(());
    }
    fn pop(&mut self) -> Result <T, String> {
        if self.cima == 0 {
            return Err(String::from("Pila vacía"));
        }
        let value = self.a[self.cima - 1]; // me guardo en esta var el
valor que ponga en la cima
        self.cima -= 1; // quito uno a la cima
        Ok(value) //devuelvo el numero
    }
}
```

• COLA genérica

Pensar en una cola de gente: los que van llegando nuevos se van añadiendo al final.

```
// len es la cantidad de valores que hay en la cola. Inicialmente 0.
// first es la posición del array en la que está el elemento que primero se
añadió a la cola. Inicialmente es 0, si bien inicialmente no hay ningún
elemento en la cola.
// last es la posición del array en la que se añadirá un nuevo elemento.
Inicialmente es 0.

#[derive(Debug, PartialEq)]
// recordar poner el tipo de la constante!!!!
struct Cola<T, const SIZE: usize> {
    a: [T; SIZE],
    first: usize,    // al trabajar con arrays, índice o logitud: USIZE.
    last: usize,
    len: usize
}

// ojo con la sintaxis GENÉRICA
impl <T: Copy + Default, const SIZE: usize> Cola<T, SIZE> {
    //const MSG_ERR: &str = "Cola llena";

    fn new() -> Cola<T, SIZE> { // llamar a la funcion default del tipo T
        Cola { a: [T::default(); SIZE], first: 0, last: 0, len: 0 } // el tipo
por el que sustituyamos T tiene que tener un constructor por defecto.
    }
    fn len(&self) -> usize {
        self.len
    }
    fn enqueue(&mut self, value: T) -> Result<(), String> {
        if self.len == SIZE {
            return Err(String::from("Cola llena"));
        }
        self.a[self.last] = value;
        self.last = if self.last == SIZE - 1 { 0 } else { self.last + 1 };
        self.len += 1;
        Ok(())
    }
    fn dequeue(&mut self) -> Result<T, String> {
        if self.len == 0 {
            return Err(String::from("Cola vacía"));
        }
        let value = self.a[self.first];
        self.first = if self.first == SIZE - 1 { 0 } else { self.first + 1 };
        self.len -= 1;
        Ok(value)
    }
}
```

